BẢNG PHÂN CÔNG NHIỆM VỤ VÀ KẾ HOẠCH ĐỒ ÁN

MÔN HỌC: MẠNG MÁY TÍNH (CSC10008) - HCMUS
ĐỀ TÀI: ỨNG DỤNG TRÒ CHUYỆN VÀ CHIA SẺ TỆP TIN P2P TRONG MẠNG LAN

1. THÔNG TIN CHUNG

Trường: Đại học Khoa học Tự nhiên - ĐHQG TP.HCM
Khoa: Công nghệ Thông tin | Môn học: Mạng máy tính (CSC10008)
Đề tài: Xây dựng ứng dụng trò chuyện và chia sẻ tệp tin ngang hàng trong mạng LAN
Công nghệ sử dụng: C++17, Qt6 Widgets, Windows Sockets (ws2_32), CMake, Ninja
Kho lưu trữ mã nguồn (GitHub): https://github.com/VN-DEV/HCMUS-Socket-Project

2. DANH SÁCH THÀNH VIÊN ĐĂNG KÝ

STT	Họ và Tên	Vai trò	Lĩnh vực phụ trách chính	Tỷ lệ đóng góp
1	**Nguyễn Mạnh Trí**	Thành viên	Giao thức mạng & Cơ chế Peer Discovery (UDP Broadcast, Heartbeat, Timeout)	33.3%
2	**Đỗ Duy Trường**	Thành viên	Hệ thống TCP Server & Xử lý Kết nối Tin nhắn (TCP Messaging, Framing)	33.3%
3	**Mai Hữu Trí**	Thành viên	Giao diện người dùng (Qt6 UI) & Module Truyền File Chunked 64KB (TCP)	33.3%

3. CHI TIẾT PHÂN CÔNG NHIỆM VỤ

3.1. Thành viên: Nguyễn Mạnh Trí

Mã nguồn đảm nhiệm: src/protocol/Protocol.h, src/protocol/Protocol.cpp, src/network/UdpDiscovery.h, src/network/UdpDiscovery.cpp, src/models/Peer.h, src/main.cpp.

- Thiết kế và hiện thực giao thức tầng ứng dụng: Xây dựng các hàm serialize và parse gói tin UDP HELLO, TCP AUTH, TCP MSG, TCP FILE header, hàm tiện ích split chuỗi an toàn theo ký tự '|'.
- Hiện thực module Peer Discovery (UDP): Tạo socket UDP broadcast cổng 9999 (bật SO_BROADCAST, SO_REUSEADDR), quản lý 3 luồng: Luồng phát định kỳ 3 giây (broadcastLoop), Luồng thu lắng nghe gói HELLO (listenLoop), Luồng kiểm tra timeout 10 giây (timeoutLoop).
- Quản lý trạng thái hiện diện: Cập nhật trạng thái người dùng (Online/Away) và phát broadcast tức thời khi thay đổi trạng thái.
- Khởi tạo và cấu hình ứng dụng: Điểm khởi đầu main.cpp, quản lý vòng đời mạng Winsock (WSAStartup, WSACleanup), cấu hình bộ lọc Wireshark để kiểm chứng gói tin UDP.

3.2. Thành viên: Đỗ Duy Trường

Mã nguồn đảm nhiệm: src/network/TcpServer.h, src/network/TcpServer.cpp, src/network/TcpConnection.h, src/network/TcpConnection.cpp.

- Hiện thực TCP Listening Server: Mở socket TCP (AF_INET, SOCK_STREAM), bật SO_REUSEADDR, bind cổng 5000 (tự động thử các cổng kế tiếp 5001, 5002 nếu bị trùng khi chạy nhiều client trên cùng một máy), gọi listen() và chạy Luồng đón kết nối (acceptLoop).
- Quản lý kết nối TCP (TcpConnection): Hiện thực thủ tục bắt tay xác thực hai chiều (Authentication Handshake) bằng thông điệp AUTH khi vừa kết nối.
- Xử lý Message Framing chống dính gói: Đọc dữ liệu từ socket TCP theo định dạng tiền tố độ dài MSG|length|text, đảm bảo đọc chính xác số byte nội dung, tránh hiện tượng Packet Sticking hoặc Packet Fragmentation.
- Đồng bộ đa luồng khi truyền dữ liệu: Khóa mutex bảo vệ lệnh send() trên socket TCP, đảm bảo việc gửi tin nhắn và gửi file không làm hỏng dòng dữ liệu byte stream.
- Xử lý ngắt kết nối: Nhận diện sự kiện ngắt kết nối từ peer (recv trả về 0 hoặc SOCKET_ERROR) và dọn dẹp socket, thông báo callback lên hệ thống.

3.3. Thành viên: Mai Hữu Trí

Mã nguồn đảm nhiệm: src/chat/ChatManager.h, src/chat/ChatManager.cpp, src/ui/MainWindow.h, src/ui/MainWindow.cpp, phần truyền nhận file trong TcpConnection.

- Hiện thực truyền nhận file theo khối (Chunked TCP File Transfer): Đọc file nhị phân từ đĩa, gửi header FILE|filename|filesize|, chia file thành các khối 64KB để truyền tuần tự mà không gây nghẽn RAM; bên nhận tự động lưu vào thư mục downloads/.
- Cập nhật tiến độ truyền file: Tính toán tỷ lệ phần trăm (percent = sentBytes * 100 / fileSize) và kích hoạt callback cập nhật thanh tiến trình QProgressBar trên giao diện.
- Quản lý phiên chat và đếm tin nhắn chưa đọc: Duy trì danh bạ kết nối, lưu lịch sử chat trong RAM, quản lý biến đếm số tin nhắn chưa đọc (Unread Count / Badge) khi có tin nhắn từ peer không được chọn.
- Thiết kế và lập trình giao diện Qt6 Widgets: Xây dựng giao diện 2 cột bằng QSplitter: bảng điều khiển cá nhân, danh sách online peers, khung hiển thị lịch sử chat, ô nhập tin nhắn và nút chọn file gửi; điều phối an toàn giữa luồng socket và luồng GUI Qt.

4. KẾ HOẠCH TIẾN ĐỘ THỰC HIỆN (5 TUẦN)

Tuần	Thời gian	Mục tiêu & Nhiệm vụ trọng tâm	Người phụ trách	Sản phẩm đầu ra
Tuần 1	08/11 - 15/11	Khởi tạo cấu trúc dự án, cấu hình CMake & Ninja, xây dựng bộ khung mã nguồn (skeleton) và tài liệu đặc tả giao thức.	Cả 3 thành viên	Repository GitHub chuẩn, build khung mã nguồn thành công.
Tuần 2	16/11 - 22/11	Hoàn thiện module UDP Discovery (phát broadcast 3s, lắng nghe cổng 9999, kiểm tra timeout 10s) và TCP Server (bind, listen, accept).	Nguyễn Mạnh Trí Đỗ Duy Trường	Chương trình tự động thấy nhau trong LAN và cập nhật danh sách peer.
Tuần 3	23/11 - 29/11	Hoàn thiện gửi/nhận tin nhắn qua TCP, xử lý Message Framing chống dính/cắt gói, quản lý phiên và lịch sử chat.	Đỗ Duy Trường Mai Hữu Trí	Chat 1-1 ổn định giữa các máy trong LAN hoặc đa client.
Tuần 4	30/11 - 06/12	Hiện thực truyền nhận file chunked 64KB, ghép nối giao diện đồ họa hoàn chỉnh với Qt6 Widgets (hiển thị tiến độ % và unread badge).	Mai Hữu Trí Nguyễn Mạnh Trí	Gửi nhận file dung lượng lớn thành công, giao diện tương tác mượt mà.
Tuần 5	07/12 - 14/12	Kiểm thử tải (Stress test), kiểm thử mất kết nối đột ngột, bắt gói tin nghiệm thu bằng Wireshark, quay video demo và hoàn thiện báo cáo.	Cả 3 thành viên	Báo cáo đồ án hoàn chỉnh, file log Wireshark (.pcapng), video demo.

5. NGUYÊN TẮC LÀM VIỆC VÀ CAM KẾT ĐÓNG GÓP

1. Bình đẳng trách nhiệm: Tất cả 3 thành viên có vai trò và trách nhiệm tương đương nhau (mỗi người đóng góp 33.3% khối lượng đồ án), cùng chịu trách nhiệm về chất lượng sản phẩm cuối cùng.
2. Quản lý mã nguồn chuyên nghiệp: Toàn bộ mã nguồn phát triển trên các nhánh tính năng (feature branches) riêng biệt và hợp nhất thông qua Pull Request trên GitHub.
3. Chuẩn mực tài liệu và mã nguồn: Tuân thủ quy chuẩn mã nguồn sạch (Clean Code), chú thích tiếng Việt rõ ràng, tuyệt đối không sử dụng ký tự biểu tượng cảm xúc (emoji) trong commit và tài liệu.